

Justification du Choix du Solveur HiGHS pour une Plateforme de Planification DSL → MiniZinc

27 avril 2026

Résumé

Cette section autonome justifie le choix actuel du solveur HiGHS pour une plateforme qui transforme des instances de planification `.planning` en modèles MiniZinc. L'argument principal est structurel : la formulation générée est majoritairement composée de variables binaires et de contraintes linéaires ou linéarisables, ce qui la rapproche d'un modèle MILP. La justification reste volontairement nuancée : HiGHS est le solveur le plus adapté à la formulation actuelle, sans prétendre qu'il soit universellement optimal pour toutes les variantes de problèmes de planning.

Table des matières

1	Contexte et objectif	3
2	Nature du modèle généré par la plateforme	3
2.1	Paramètres de taille et familles de variables	3
2.2	Structure des contraintes	3
3	Pourquoi la formulation actuelle favorise HiGHS	4
4	Problème de planification et formulation mathématique	4
5	Comparaison avec Gecode, Chuffed et OR-Tools CP-SAT	5
5.1	Gecode	5
5.2	Chuffed	5
5.3	OR-Tools CP-SAT	5
6	Critères de choix d'un solveur	5
7	Justification synthétique du choix de HiGHS	6
8	Limites de la justification	6
9	Conclusion	7

1 Contexte et objectif

La plateforme étudiée suit la chaîne de traitement suivante : un fichier DSL `.planning` est analysé, traduit en MiniZinc, aplati par le compilateur MiniZinc, puis transmis à un solveur. MiniZinc peut cibler plusieurs solveurs, notamment HiGHS, Gecode, Chuffed, OR-Tools CP-SAT, CBC, SCIP, Gurobi et CPLEX, selon la configuration disponible [1, 2].

Tous ces solveurs ne sont pas équivalents sur une même instance. Le point clé est le suivant :

Le choix du solveur ne dépend pas uniquement de la nature générale du problème de planification, mais aussi de la manière dont ce problème est représenté dans MiniZinc.

L’objectif de cette section est donc d’établir, avec un raisonnement explicite, pourquoi HiGHS apparaît actuellement comme le solveur le plus cohérent avec la formulation produite par la plateforme.

2 Nature du modèle généré par la plateforme

2.1 Paramètres de taille et familles de variables

On introduit les paramètres :

A = nombre d’activités, R = nombre de ressources, K = nombre de rôles, T = nombre de slots t

Les variables majeures du modèle sont typiquement :

- `start_time[a]` : variable entière (ou discrète) de début de l’activité a ;
- `assignment[a,r]` : booléen d’affectation activité–ressource ;
- `role_assignment[a,k,r]` : booléen d’affectation activité–rôle–ressource.

Proposition 2.1 (Comptage des variables principales). *Sous cette modélisation,*

$$N_{start} = A, \quad N_{assign} = A \times R, \quad N_{role} = A \times K \times R,$$

et

$$N_{variables} = A + A \times R + A \times K \times R.$$

Démonstration. Le résultat suit directement du nombre d’indices nécessaires pour instancier chaque famille de variables : un indice pour `start_time`, deux indices pour `assignment`, et trois indices pour `role_assignment`. La somme donne le total. $\square$

2.2 Structure des contraintes

La formulation contient principalement :

- contraintes de cardinalité (sommes de booléens),
- contraintes de capacité (inégalités linéaires),
- contraintes de non-chevauchement (souvent décomposées),
- contraintes de disponibilité,

- contraintes de précédence,
- implications logiques,
- éventuellement une fonction objectif additive (pénalités/préférences).

Cette structure produit un modèle dominé par :

1. des décisions binaires,
2. des agrégations linéaires de booléens,
3. des contraintes linéaires ou linéarisables,
4. un objectif fréquemment linéaire.

Remarque 2.1. *Même si le problème métier est un problème de planification combinatoire, la représentation mathématique actuelle est proche d'une formulation MILP.*

3 Pourquoi la formulation actuelle favorise HiGHS

HiGHS est un solveur d'optimisation linéaire open source, conçu pour les problèmes LP/MIP (et QP) [4]. Dans l'écosystème MiniZinc, il est disponible comme backend dédié [2, 3].

L'adéquation avec la formulation actuelle s'explique par :

- la forte proportion de variables binaires ;
- des contraintes de cardinalité exprimables comme sommes de binaires ;
- des contraintes de capacité naturellement linéaires ;
- une fonction objectif souvent additive ;
- des implications logiques fréquemment linéarisables.

HiGHS est pertinent non pas parce que le problème de planification est simple, mais parce que la formulation MiniZinc produite par la plateforme se rapproche d'un modèle linéaire en variables binaires.

4 Problème de planification et formulation mathématique

Un même problème de planning peut être encodé selon plusieurs paradigmes :

1. programmation par contraintes (CP),
2. SAT/CP-SAT,
3. MILP,
4. hybride.

Ainsi, le solveur optimal dépend du *modèle effectivement généré* :

- si la formulation exploite fortement des contraintes globales de scheduling, un solveur CP peut être très pertinent ;
- si la formulation est fortement logique/booléenne, CP-SAT peut devenir très compétitif ;
- si la formulation est majoritairement binaire + linéaire, un solveur MILP tel que HiGHS est naturellement cohérent.

5 Comparaison avec Gecode, Chuffed et OR-Tools CP-SAT

5.1 Gecode

Gecode est un solveur CP orienté domaines finis, avec un support natif de nombreuses contraintes globales dans MiniZinc [1, 5]. Il peut être excellent sur des modèles de scheduling exprimés en contraintes globales riches.

Dans la formulation actuelle, il peut être moins naturellement favorisé si :

- les contraintes sont déjà très décomposées ;
- le non-chevauchement est formulé principalement en paire-à-paire ;
- la structure linéaire domine la structure CP globale.

Conclusion nuancée : Gecode peut être performant sur certains modèles de planning, mais il est moins naturellement aligné avec une formulation majoritairement linéaire et booléenne.

5.2 Chuffed

Chuffed est un solveur CP à *lazy clause generation*, combinant propagation CP et apprentissage de clauses [1, 6]. Il est souvent pertinent sur des modèles logiques/combinatoires denses.

Il reste un candidat sérieux à évaluer, en particulier lorsque les implications logiques sont centrales. Cependant, si l'essentiel du modèle est représenté par des sommes linéaires de binaires et un objectif linéaire, HiGHS peut garder un avantage structurel.

Conclusion nuancée : Chuffed est un candidat crédible à comparer expérimentalement, mais HiGHS reste plus cohérent avec la structure linéaire actuelle.

5.3 OR-Tools CP-SAT

OR-Tools CP-SAT est un solveur hybride très performant sur de nombreux problèmes entiers et booléens, avec capacités de raisonnement logique et contraintes linéaires sur entiers [7, 8].

Il peut donc être très pertinent pour la plateforme. Néanmoins, son efficacité dépend de la forme exacte du modèle aplati, de l'intégration MiniZinc et du compromis entre raisonnement logique et exploitation de relaxations linéaires.

Conclusion nuancée : OR-Tools CP-SAT est probablement l'un des meilleurs solveurs à tester en complément de HiGHS, surtout si la formulation évolue vers davantage de logique ou de scheduling global.

6 Critères de choix d'un solveur

On peut retenir les critères suivants :

- nature des variables (continues/entières/binaires),
- linéarité des contraintes,
- présence et nature de l'objectif,
- proportion de booléens,

- présence de contraintes globales,
- capacité à prouver l’optimalité,
- temps de résolution et robustesse,
- disponibilité MiniZinc,
- simplicité d’intégration,
- contraintes de licence,
- stabilité sur des familles d’instances variées.

Pour la formulation actuelle, HiGHS est favorable sur plusieurs critères : open source, intégration MiniZinc, cohérence LP/MIP, efficacité attendue sur contraintes linéaires et variables binaires.

7 Justification synthétique du choix de HiGHS

Le raisonnement peut être résumé ainsi :

1. la plateforme génère un modèle MiniZinc ;
2. le modèle contient de nombreuses variables booléennes ;
3. les contraintes sont majoritairement linéaires ou linéarisables ;
4. l’objectif éventuel est additif ;
5. l’ensemble correspond à une structure de type MILP ;
6. HiGHS est conçu pour LP/MIP ;
7. HiGHS est donc le solveur le plus cohérent avec la formulation actuelle ;
8. d’autres solveurs restent pertinents si la formulation change.

Le choix de HiGHS est donc justifié par l’adéquation entre la structure du modèle généré et la classe de problèmes pour laquelle HiGHS est conçu.

8 Limites de la justification

La conclusion doit rester conditionnelle :

- HiGHS n’est pas nécessairement le meilleur solveur pour toutes les formulations de planning ;
- un solveur CP peut devenir supérieur si les contraintes globales sont davantage exploitées ;
- OR-Tools CP-SAT peut surpasser HiGHS sur des formulations très logiques ;
- Gurobi/CPLEX peuvent être plus performants sur certaines grandes instances MILP ;
- la décision finale doit être validée expérimentalement sur les mêmes instances `.planning`.

9 Conclusion

Dans l'état actuel de la plateforme, HiGHS apparaît comme le solveur le plus adapté, car la formulation MiniZinc générée est proche d'un MILP dominé par des variables binaires et des contraintes linéaires/linéarisables.

Ce résultat doit toutefois être lu comme une conclusion *contextuelle* : Gecode, Chuffed, OR-Tools CP-SAT, CBC, SCIP, Gurobi et CPLEX restent des alternatives pertinentes selon l'évolution de la formulation, la taille des instances et les contraintes d'intégration.

La formulation la plus rigoureuse est donc :

HiGHS apparaît comme le choix le plus approprié pour la formulation MiniZinc actuelle de la plateforme.

Références

- [1] MiniZinc Handbook, *Solving Technologies and Solver Backends*, <https://docs.minizinc.dev/en/latest/solvers.html>.
- [2] MiniZinc, *Downloads and Available Solvers*, <https://www.minizinc.org/downloads/>.
- [3] MiniZinc Handbook, *Change Log (HiGHS interface in MiniZinc)*, <https://docs.minizinc.dev/en/2.9.0/changelog.html>.
- [4] HiGHS, *High-performance linear optimization software*, <https://highs.dev/>.
- [5] Gecode, *Constraint solving toolkit*, <https://www.gecode.dev/>.
- [6] Chuffed, *The Chuffed CP solver*, <https://github.com/chuffed/chuffed>.
- [7] Google OR-Tools, *CP-SAT Solver*, https://developers.google.com/optimization/cp/cp_solver.
- [8] Google OR-Tools, *MPSolver Interface*, <https://developers.google.com/optimization/lp/mpsolver>.
- [9] COIN-OR, *Introduction to CBC*, <https://coin-or.github.io/Cbc/intro.html>.
- [10] SCIP, *SCIP Optimization Suite*, <https://www.scipopt.org/>.
- [11] Gurobi, *Gurobi Optimizer Reference Manual*, <https://docs.gurobi.com/projects/optimizer/en/current/index.html>.
- [12] IBM, *CPLEX Optimizer*, <https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-optimizer>.